Module: Testing, Debugging & Clean Code

Previous module: React Hooks / API Deploy
Next module: Career Resume Interview

Module Outcome:

- Write a unittest that fails first, then proves the fix
- Read a traceback and fix the line — not the symptom
- Leave code a teammate can review: names, small functions, no mystery `x`

Why testing and clean code?

In Indian IT interviews and on the job, you will:

- Fix bugs in legacy Python/Django code
- Submit PRs (pull requests) reviewed by seniors
- Be asked "did you test?" in standup



Where used?
- **Swiggy** — discount / GST helpers must not ship wrong ₹ amounts
- College apps — marks averages and login validators covered by unit tests
- **HDFC Bank** / **Employee Management Portal** — money math needs tests before production merges
- Service companies — PR reviews expect tests + clear function names
- Capstone — prove CRUD edge cases (empty list, bad percent) before demo

Testing proves your code works before manager finds bugs. **Clean code** helps teammates read your work at 6 PM before release.

Debugging vs guessing

Print debugging: Fast for tiny scripts — `print("here", x)` — but messy in production.

VS Code debugger: Pause program at line, inspect variables, step line by line — professional approach.

Logging: Structured messages with levels (DEBUG, INFO, WARNING, ERROR) saved to file or console — used in real Django projects.

Clean code in one sentence

Write code that reads like clear instructions for the next developer — often **you**, three months later.

Concepts

VS Code debugging flow

1. Open Python file

2. Click gutter to set **breakpoint** (red dot)
3. Run → **Start Debugging** (F5) → Python File
4. When paused: **Variables** panel, **Watch**, Step Over (F10), Continue (F5)

`launch.json` snippet (optional):

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal"
    }
  ]
}
```

Set a breakpoint on the `return` line in `format_inr` and run the file in VS Code debug mode to inspect variables.

```
def format_inr(amount):
    # Set breakpoint on the next line in VS Code
    return f"₹{amount:,.2f}"

if __name__ == "__main__":
    print(format_inr(1250))
    print(format_inr(90))
```

Expected Output:

```
₹1,250.00
₹90.00
```

Follow the VS Code debug checklist to set a breakpoint, launch Python File debug, and step through with F10.

```
# VS Code debug

1. Open format_inr.py
2. Click left of line number (red breakpoint)
3. Run and Debug → Python File
4. Step Over (F10) and watch Variables
```

Logging levels

Level	When
DEBUG	Detailed diagnostic (dev)
INFO	Normal events (user logged in)
WARNING	Unexpected but handled
ERROR	Failure needing attention

Level	When
CRITICAL	System down

Note:
- `logger.debug` with INFO level → missing logs — set the level correctly or use INFO.

Configure INFO-level logging and emit info, warning, and error messages for a campus discount scenario.

```
import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
logging.info("Starting CampusCart discount calc")
logging.warning("Price missing — using 0")
logging.error("Cannot connect to MySQL")
```

Expected Output:

```
INFO: Starting CampusCart discount calc
WARNING: Price missing — using 0
ERROR: Cannot connect to MySQL
```

Add DEBUG logging inside `apply_discount` and call it to see price and percent values in the log output.

```
import logging

logging.basicConfig(level=logging.DEBUG)

def apply_discount(price, percent):
    logging.debug("price=%s percent=%s", price, percent)
    return price * (1 - percent / 100)

print(apply_discount(100, 10))  # 90.0
```

Expected Output:

```
DEBUG:root:price=100 percent=10
90.0
```

Remember: Prefer asserting **return values** in tests — capturing `print` output is brittle.

Common Error: Naming the file `discount_test.py` instead of `test_discount.py` → pytest collects **0** tests.

Interview: unittest (stdlib, class-based) vs pytest (simple `assert`, popular in startups) — both prove functions; pick one and run it in CI.

unittest vs pytest

Feature	unittest	pytest
Style	Class-based	Function-based common
Assertions	many assertX methods	plain `assert`
Running	`python -m unittest`	`pytest`
Industry	stdlib, Django default	Very popular in startups

Note:
- Testing print output only → fragile tests — assert return values.
- No test for edge cases → production crash on empty list — test `[]`, zero, negative.
- Wrong test file discovery → 0 tests collected — name files `test_*.py` (add `__init__.py` if needed).
- Mutating shared test data → order-dependent failures — use fresh fixtures per test.

Write pytest-style tests for `apply_discount` covering a valid ten-percent case and a `ValueError` for a bad percent.

```
# test_discount.py
def apply_discount(price, percent):
    if percent < 0 or percent > 100:
        raise ValueError("bad percent")
    return price * (1 - percent / 100)

def test_ten_percent():
    assert apply_discount(100, 10) == 90

def test_bad_percent():
    import pytest
    with pytest.raises(ValueError):
        apply_discount(100, -1)
```

Write a unittest `TestCase` class for `add_gst` and run it with `unittest.main()`.

```
import unittest

def add_gst(amount, rate=0.05):
    return round(amount * (1 + rate), 2)

class TestGst(unittest.TestCase):
    def test_thali(self):
        self.assertEqual(add_gst(90), 94.5)

if __name__ == "__main__":
    unittest.main()
```

DRY and small functions

Bad: One 200-line `main()` with copy-pasted validation.

Better: `validate_email()`, `calculate_gst()`, `format_inr()` — each tested separately.

Note:
- Giant functions → untestable — extract helpers.
- Over-DRY too early → confusing abstractions — follow the rule of three.

Extract a `format_inr` helper and use it inside `line_total` to format quantity × price as rupees.

```
def format_inr(n):
    return f"₹{n:,.2f}"

def line_total(qty, price):
    return format_inr(qty * price)

print(line_total(2, 60))  # ₹120.00
```

Document the before/after benefit of replacing copy-pasted GST logic with one shared `add_gst` function.

```
# Before: copy-paste GST in 4 places
# After: one add_gst(amount) used everywhere
# Benefit: fix rounding once — all screens update
```

Meaningful commits

Good: `Fix negative price validation in ProductSerializer`
Bad: `changes`, `final`, `final2`, `done`

Note:
- Commit everything Friday → unreviewable PR — prefer small daily commits.

List three good commit message examples that describe the change instead of generic labels like "final" or "done".

```
Add pytest for apply_discount edge cases
Fix logging format to include levelname
Update README with pytest command
```

Run pytest, then stage and commit GST helpers only after tests pass green.

```
pytest -q
git add utils/ tests/
git commit -m "Add GST helpers with failing-then-passing tests"
```

Step-by-Step Annotated Code

Step 1 — Sample module to test

`utils/discount.py`:

```
"""Discount helpers for training lab."""

def calculate_discount(price: float, percent: float) -> float:
    """Return final price after percent discount. Raises ValueError if invalid."""
    if price < 0:
        raise ValueError('price must be non-negative')
```

```
    if percent < 0 or percent > 100:
        raise ValueError('percent must be 0–100')
    return round(price * (1 - percent / 100), 2)

def format_inr(amount: float) -> str:
    """Format amount as Indian rupees string."""
    return f'₹{amount:,.2f}'
```

Step 2 — unittest

`tests/test_discount_unittest.py`:

```
import unittest
from utils.discount import calculate_discount, format_inr

class TestDiscount(unittest.TestCase):
    def test_ten_percent_off(self):
        # Arrange
        price = 1000
        # Act
        result = calculate_discount(price, 10)
        # Assert
        self.assertEqual(result, 900.0)

    def test_invalid_percent_raises(self):
        with self.assertRaises(ValueError):
            calculate_discount(100, 150)

    def test_format_inr(self):
        self.assertEqual(format_inr(1299.5), '₹1,299.50')

if __name__ == '__main__':
    unittest.main()
```

Run: `python -m unittest tests/test_discount_unittest.py`

Step 3 — pytest equivalent

`tests/test_discount_pytest.py`:

```
import pytest
from utils.discount import calculate_discount, format_inr

def test_ten_percent_off():
    assert calculate_discount(1000, 10) == 900.0

def test_invalid_percent_raises():
    with pytest.raises(ValueError):
        calculate_discount(100, 150)

@pytest.mark.parametrize('amount,expected', [
    (100, '₹100.00'),
    (1299.5, '₹1,299.50'),
])
def test_format_inr(amount, expected):
    assert format_inr(amount) == expected
```

Run: `pytest -v`

Step 4 — Logging example

Use `logging.getLogger(__name__)` with INFO for normal flow and ERROR for invalid input — replace stray `print()` in views.

Step 5 — Refactor for clean code

Extract `line_total`, `calculate_subtotal`, `calculate_gst`, and `calculate_grand_total` from one monolithic `report()` function — each gets its own pytest.

Step 6 — Debugging exercise

Buggy code:

```
def average_marks(marks):
    total = 0
    for m in marks:
        total += m
    return total / len(marks) - 1   # BUG: wrong formula

print(average_marks([80, 90, 100]))
```

Set breakpoint before return; inspect `total`, `len(marks)`; fix to `total / len(marks)`.

Practice Exercises

Basic

1. Debug script with off-by-one loop; document breakpoint screenshot.
   Model answer:
   Set a breakpoint on the loop line in VS Code/PyCharm, run under debugger, watch the index go past the last valid index. Fix by using `range(len(items))` correctly or iterating `for item in items`. Capture the Variables pane showing the bad index.
2. Convert 5 prints to logging with INFO and ERROR.
   Model answer:

```
import logging
logging.basicConfig(level=logging.INFO)
log = logging.getLogger(__name__)
log.info("Starting job")
try:
    ...
except Exception:
    log.exception("Job failed")  # ERROR + traceback
```

3. Write 2 unittest cases for `is_valid_email` (simple `@` check).
   Model answer:

```
import unittest

def is_valid_email(s):
    return isinstance(s, str) and "@" in s

class TestEmail(unittest.TestCase):
    def test_valid(self):
```

```
        self.assertTrue(is_valid_email("a@b.com"))
    def test_invalid(self):
        self.assertFalse(is_valid_email("no-at-sign"))
```

Intermediate

1. pytest parametrize 5 inputs for GST calculation.
   Model answer:

```
import pytest

def gst(amount, rate=0.18):
    return round(amount * rate, 2)

@pytest.mark.parametrize("amount,expected", [
    (100, 18.0), (0, 0.0), (50, 9.0), (199.99, 36.0), (1, 0.18),
])
def test_gst(amount, expected):
    assert gst(amount) == expected
```

2. Refactor 40-line Django view function — extract validation helper.
   Model answer:
   Move field checks into `validate_payload(data) -> errors dict` (or raise `ValidationError`). The view then: parse POST → call helper → if errors, re-render; else save and redirect. Shorter view, reusable validation.
3. Rewrite 3 bad commit messages into proper format.
   Model answer:
   Bad: `update`, `fix`, `asdf` → Good: `Fix login redirect after successful auth`, `Add GST helper with unit tests`, `Refactor task delete to require POST and ownership`.

Interview-style

1. Unit vs integration test?
   Model answer:
   Unit tests check one function/class in isolation (often with mocks). Integration tests check collaborating pieces (DB, API, views) working together.
2. What is TDD (high level)?
   Model answer:
   Test-Driven Development: write a failing test first, implement the minimum code to pass, then refactor — red → green → refactor.
3. Why meaningful variable names matter?
   Model answer:
   Clear names document intent so teammates (and future you) understand code without decoding abbreviations. They reduce bugs and speed reviews more than clever one-letter names.

Mini-Project (Assignment): Tested Utils Package

Lab kit (build here): 15_Lab_Assets/26_unittest_utils/

What to build

- `utils/` with at least 3 pure functions (discount, tax, string helper)
- 8+ pytest tests covering normal and error paths

- `logging` in one script
- README with `pytest` command and sample git log (3 good commits)

Stretch

- Add a VS Code `launch.json` debug config for one failing test.
- Cover GST inclusive vs exclusive with two tests.

Done when

- [ ] Lab kit tests-catch-the-bug ticket completed
- [ ] `pytest` is green locally
- [ ] At least 3 pure utils + 8 tests
- [ ] README shows how a teammate runs tests

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. When is print debugging acceptable on job?
2. Can you have too many tests? What is pragmatic for a course project?
3. how is code review similar to audit checklist in finance?
4. DRY vs "copy code twice before abstracting" — tradeoff?
5. Why do Indian companies ask for GitHub with commits not one zip upload?

Q1. What is unit testing?
A: Testing individual functions/units in isolation with known inputs and expected outputs.

Q2. unittest vs pytest preference?
A: pytest for concise syntax and plugins; unittest built-in; both valid; know one well.

Q3. How do you debug Python in IDE?
A: Breakpoints, step through, inspect variables, watch expressions, read stack trace.

Q4. What is clean code?
A: Readable, maintainable code with clear names, small functions, minimal duplication, consistent style.

Q5. Why logging over print?
A: Levels, formatting, handlers to files, can disable debug in production without removing statements.
